MA388 Sabermetrics: Lesson 22

Simulating Baseball: Part II

Lesson Objectives

1. Use the Bradley-Terry model to compute the probability one team beats another.
2. Build a Monte Carlo simulation of an entire baseball season, including schedule generation.
3. Compare simulated season results to the underlying talent levels of teams.
4. Generalize the Monte Carlo framework to simulate other probabilistic events.

Simulating a Game

In the last lesson, we simulated the number of runs in a half-inning by modeling individual at-bats and state transitions. Now we're going to simulate a whole season. Instead of simulating each at-bat, we'll start our simulation efforts at the level of modeling individual game outcomes.

Let's come up with some ideas for deciding how to predict the winner of a game.

The Bradley-Terry model states the probability of Team A winning against Team B with this logistic function:

$$P(A \text{ wins}) = \frac{\exp(T_A)}{\exp(T_A) + \exp(T_B)}$$

where T_A and T_B are the talent levels associated with Teams A and B, respectively. Now consider a different formulation provided by Bill James in the early 1980s:

$$P(A \text{ wins}) = \frac{P_A/(1-P_A)}{P_A/(1-P_A) + P_B/(1-P_B)}$$

where P_A and P_B are the winning percentages of Teams A and B, respectively. How are these models related?

So who wins in a game between Team A and B? Let Team A have $P_A = 0.55$ and Team B have $P_B = 0.48$. (Note: These don't have to add to 1!)

```
P_A = 0.55
P_B = 0.48
P_A_wins = (P_A / (1 - P_A)) / ((P_A / (1 - P_A)) + (P_B / (1 - P_B)))
P_B_wins =  (P_B / (1 - P_B)) / ((P_B / (1 - P_B)) + (P_A / (1 - P_A)))
P_A_wins
```

```
[1] 0.5697211
```

Great, Team A wins with probability 0.57. Unfortunately, we don't need a probability of winning – we need a winner! How can we produce a winner by knowing that Team A is expected to win with probability 0.57?

Here are two ways we might think about declaring a winner.

1. Draw a Uniform(0,1) random number. If the probability is less than or equal to 0.57, Team A wins. If it's greater than 0.57, Team B wins.

```
set.seed(388)
rand <- runif(n = 1, min = 0, max = 1)
rand
```

```
[1] 0.4489612
```

```
ifelse(rand < P_A_wins, "Team A wins!", "Team B wins!")
```

```
[1] "Team A wins!"
```

2. Alternatively, we can flip a coin with other than 50-50 probabilities of heads and tails.

```
A_vs_B_outcome <- rbinom(n = 1, size = 1, prob = P_A_wins)
ifelse(A_vs_B_outcome == 1, "Team A wins!", "Team B wins!")
```

```
[1] "Team A wins!"
```

These happen to agree this time, but there's no requirement for that to be the case. We effectively flipped two coins here and could have gotten different outcomes.

Simulating a Season

If we're going to simulate a season, we first need to create a schedule. Suppose there were three teams (A, B, and C), and every team plays every team once at home and once away.

- A vs. B @ A
- A vs. B @ B
- A vs. C @ A
- A vs. C @ C
- B vs. C @ B
- B vs. C @ C

```
library(tidyverse)
library(Lahman)
library(knitr)

teams <- c("A", "B", "C")
k = 1 # number of times each team hosts each other team
num_teams <- length(teams)
Home <- rep(rep(teams, each = num_teams), k)
Visitor <- rep(rep(teams, num_teams), k)
tibble(Home = Home, Visitor = Visitor) |>
  filter(Home != Visitor) |>
  kable()
```

Home	Visitor
A	B
A	C
B	A
B	C
C	A
C	B

Since we have code for a valid schedule for three teams playing each other once at home and once away, we can feel good about extending this to more complicated seasons with more teams and more games between pairs of teams. Suppose we have 20 teams, each playing every other team three times each, both at home and away. That should be:

$$\frac{20 \text{ teams} \times 19 \text{ opponents} \times 6 \text{ games per pair of teams}}{2 \text{ teams per game}} = 1140 \text{ games}$$

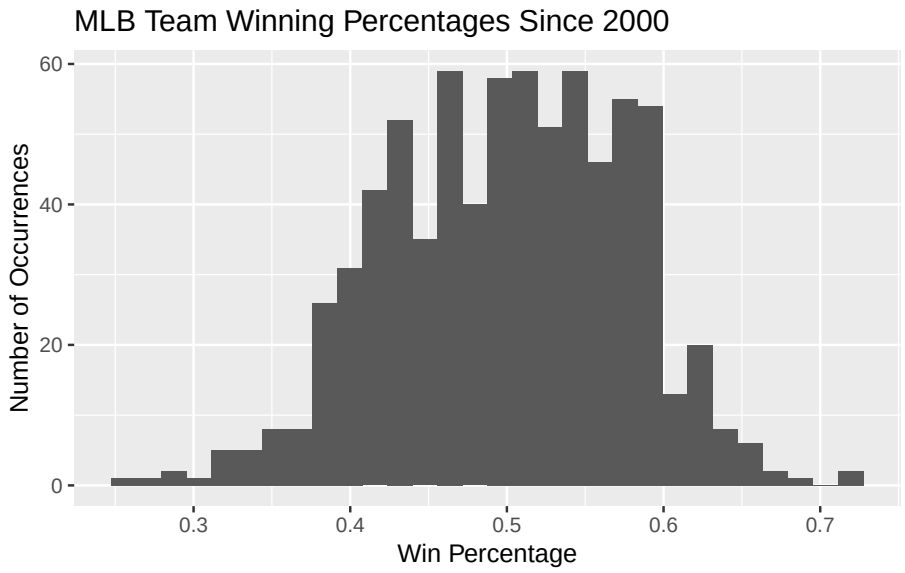
```
teams <- LETTERS[1:20]
k = 3 # number of times each team hosts each other team
num_teams <- length(teams)
Home <- rep(rep(teams, each = num_teams), k)
Visitor <- rep(rep(teams, num_teams), k)
schedule <- tibble(Home = Home, Visitor = Visitor) |>
  filter(Home != Visitor)
schedule |> nrow()
```

```
[1] 1140
```

Simulating Talent

Let's consider the win-loss records of the last ~20 years. There will be some amazing teams and some not-so-hot teams.

```
Teams |>
  select(yearID, name, W, G) |>
  filter(yearID >= 2000) |>
  mutate(win_perc = W / G) |>
  arrange(-win_perc) |>
  ggplot(aes(x = win_perc)) +
  geom_histogram() +
  labs(title = "MLB Team Winning Percentages Since 2000",
       x = "Win Percentage",
       y = "Number of Occurrences")
```




```
Teams |>
  filter(yearID==2025)
```

```
 [1] yearID         lgID           teamID         franchID       divID
 [6] Rank           G              Ghome          W              L
[11] DivWin         WCWin          LgWin          WSWin          R
[16] AB             H              X2B            X3B            HR
[21] BB             SO             SB             CS             HBP
[26] SF             RA             ER             ERA            CG
```

```
[31] SHO            SV             IPouts         HA             HRA
[36] BBA            SOA            E              DP             FP
[41] name           park           attendance     BPF            PPF
[46] teamIDBR       teamIDlahman45 teamIDretro
<0 rows> (or 0-length row.names)
```

Let's sample 20 winning percentages from these historic teams to serve as the "talent" level of the teams in our simulated league.

```
talent <- Teams |>
  filter(yearID >= 2000) |>
  mutate(win_perc = W / G) |>
  pull(win_perc) |>
  sample(20)
talent |> round(3)
```

```
 [1] 0.562 0.599 0.494 0.519 0.481 0.549 0.506 0.432 0.494 0.512 0.475 0.483
[13] 0.611 0.586 0.494 0.512 0.556 0.519 0.562 0.552
```

Now we need to assign talent scores to our 20 teams.

```
teams <- data.frame(name = LETTERS[1:20], talent = talent)

schedule <- schedule |>
  left_join(teams, join_by(Home == name)) |>
  rename(talent_home = talent) |>
  left_join(teams, join_by(Visitor == name)) |>
  rename(talent_visitor = talent)

head(schedule) |>
  kable(digits = 3)
```

Home	Visitor	talent_home	talent_visitor
A	B	0.562	0.599
A	C	0.562	0.494
A	D	0.562	0.519
A	E	0.562	0.481
A	F	0.562	0.549
A	G	0.562	0.506

Let's add the probability that the home team wins using Bill James' formula:

```
schedule <- schedule |>
  mutate(P_Home_wins = (talent_home / (1 - talent_home)) /
           ((talent_home / (1 - talent_home)) +
              (talent_visitor / (1 - talent_visitor))))
```

Finally, we're ready to determine game winners.

```
schedule <- schedule |>
  mutate(outcome = rbinom(nrow(schedule), 1, P_Home_wins),
         Winner = if_else(outcome == 1, Home, Visitor)
  ) |>
  select(-outcome)

head(schedule) |>
  kable(digits = 3)
```

Home	Visitor	talent_home	talent_visitor	P_Home_wins	Winner
A	B	0.562	0.599	0.462	A
A	C	0.562	0.494	0.568	C
A	D	0.562	0.519	0.543	D
A	E	0.562	0.481	0.580	A
A	F	0.562	0.549	0.512	F
A	G	0.562	0.506	0.556	G

Season Standings

```
standings <- schedule |>
  summarize(Wins = n(),.by='Winner') |>
  arrange(-Wins) |>
  rename(Team = Winner)
standings |>
  kable()
```

Team	Wins
P	74
M	69
B	68
S	66
A	64

Team	Wins
F	64
R	63
J	61
D	59
C	55
Q	54
O	53
N	52
I	52
T	51
E	49
L	48
G	47
H	46
K	45

How do the season results align with the original talent rankings? Did any teams significantly beat or fail to live up to expectations?

```
teams |>
  left_join(standings, join_by(name == Team)) |>
  arrange(-talent) |>
  mutate(talent_rank = row_number()) |>
  arrange(-Wins) |>
  mutate(league_rank = row_number()) |>
  kable(col.names = c("Team", "Talent", "Wins", "Talent Rank", "League Rank"),
        digits = 3)
```

Team	Talent	Wins	Talent Rank	League Rank
P	0.512	74	12	1
M	0.611	69	1	2
B	0.599	68	2	3
S	0.562	66	5	4
A	0.562	64	4	5
F	0.549	64	8	6
R	0.519	63	10	7
J	0.512	61	11	8
D	0.519	59	9	9
C	0.494	55	14	10
Q	0.556	54	6	11
O	0.494	53	16	12
N	0.586	52	3	13
I	0.494	52	15	14
T	0.552	51	7	15
E	0.481	49	18	16
L	0.483	48	17	17
G	0.506	47	13	18
H	0.432	46	20	19
K	0.475	45	19	20

Monte Carlo Simulation Keys

1. Model the probabilistic event(s). In our case, this was the outcome of a single game based on two teams' talent levels.
2. Replicate every feature of the system, pipeline, process, etc., as closely as possible to reality.
3. Repeat the simulation hundreds or thousands of times to observe the long-run behavior of the system.

Example: In the notional card game *High-3*, every player is dealt five cards. They sum the three highest cards, and the highest total wins. Cards 2 through 10 are worth their face value. Face cards are worth 10, and aces are worth 1. Thus, the lowest possible score is 4 (A-A-A-A-2), and the highest possible score is 30 (many ways to have three 10s or face cards.)

```
deck <- rep(c(2:10,"J","Q","K","A"), 4)
deck
```

```
 [1] "2"  "3"  "4"  "5"  "6"  "7"  "8"  "9"  "10" "J"  "Q"  "K"  "A"  "2"  "3"
[16] "4"  "5"  "6"  "7"  "8"  "9"  "10" "J"  "Q"  "K"  "A"  "2"  "3"  "4"  "5"
[31] "6"  "7"  "8"  "9"  "10" "J"  "Q"  "K"  "A"  "2"  "3"  "4"  "5"  "6"  "7"
[46] "8"  "9"  "10" "J"  "Q"  "K"  "A"
```

```
values <- rep(c(2:10,10,10,10,1), 4)
values
```

```
 [1]  2  3  4  5  6  7  8  9 10 10 10 10  1  2  3  4  5  6  7  8  9 10 10 10 10
[26]  1  2  3  4  5  6  7  8  9 10 10 10 10  1  2  3  4  5  6  7  8  9 10 10 10
[51] 10  1
```

1. Model the probabilistic event. (Draw a 5-card hand.)

```
hand <- sample(values, 5, replace = FALSE)
hand
```

```
[1] 10 10  2  1  5
```

2. Replicate the system or process. (Determine the score of this hand.)

```r
score <- sort(hand, decreasing = TRUE) |> _[1:3] |> sum()
score
```

```
[1] 25
```

3. Repeat the system many times.

```r
num_trials <- 100000
scores <- rep(0, num_trials)
for(i in seq_along(scores)){
  hand <- sample(values, 5, replace = FALSE)
  scores[i] <- sort(hand, decreasing = TRUE) |> _[1:3] |> sum()
}
hist(scores, breaks = 4:30,
     main = "Histogram of High-3 Scores",
     xlab = "Scores")
```